GATE
CS & DA

Artificial Intelligence

Informed Search

By- SIDDHARTH SABHARWAL SIR

Lecture - 02

Recap of Previous Lecture

- Topic
- Topic
- Topic
- Topic
- Topic

A*

BFS-DFS

Test DPP

⇒ Video sol in Telegram group

⇒ Siddharth Sir AI/ML

Topics to be Covered

- Topic — Type of Heuristic
- Topic — A* theory
- Topic — AO* algorithm
- Topic
- Topic

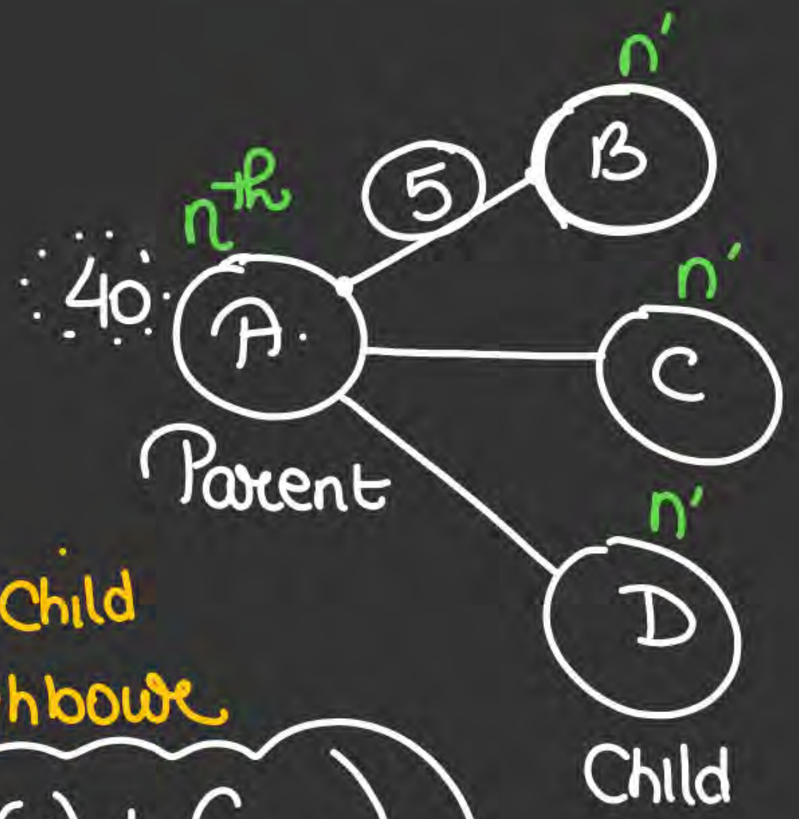
Heuristic value ⇒ Value @ any node = Cost to reach target node from 'n' th node

→ Two type

→ admissible → @ any node if heuristic value ≤ the actual cost to move from from node to goal.

→ non admissible → if heuristic value > the actual cost

Heuristic Values are Consistent



⇒ Parent - Child or neighbour

$$h(n) \leq \left(h(n') + C_{n-n'}\right)$$

Type of Heuristic values

1. In the context of search algorithms, especially in artificial intelligence and pathfinding (like A* algorithm), a heuristic function helps estimate the cost of reaching the goal from a given state. There are two important properties that a heuristic function can have: admissibility and consistency (also known as the monotonicity).

* Informed search will give optimal result if h(n) are consistent & admissible.

Type of Heuristic values

Admissible Heuristic

- A heuristic is admissible if it never overestimates the cost to reach the goal. In other words, it is always equal to or less than the true cost from the current state to the goal.
- Formally, for a heuristic h(n)h(n) to be admissible, it must satisfy: $h(n) \leq h*(n)$
- where h(n) is the heuristic cost from node ~~m~~ n to the goal, and h∗(n) is the true cost from ~~m~~ n to the goal.
- Admissibility ensures that an algorithm like A* will find the optimal solution.

Type of Heuristic values

Consistent (Monotonic) Heuristic

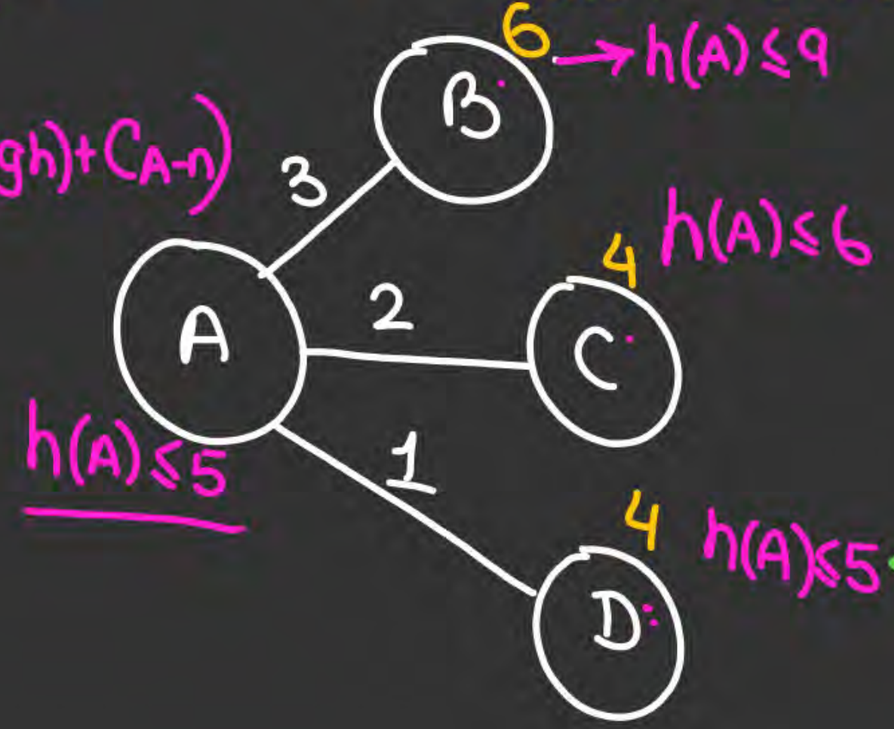
- A heuristic is consistent if the estimated cost is always less than or equal to the estimated cost from any neighboring node plus the step cost of reaching that neighbor.
- Formally, for a heuristic h(n) to be consistent, it must satisfy the following condition for every node nn and every successor n′ of n: $h(n) \leq c(n,n') + h(n')$
- where c(n,n′) is the actual cost from node n to its successor n′, and h(n′) is the heuristic cost from n′ to the goal.

Q. If heuristic are Consistent then they are admissible.

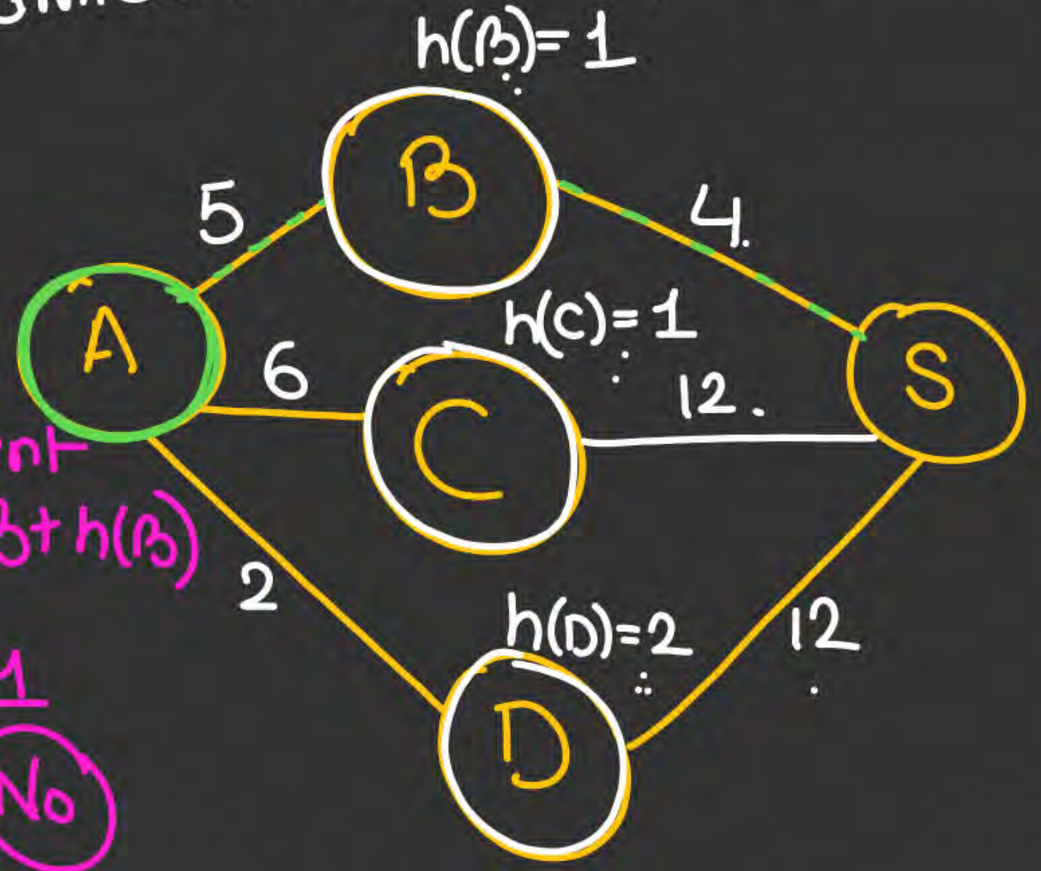
Q. False.

" " " admissible " " " Consistent.

$h(A) \le (h(neigh) + C_{A-n})$



For admissible $h(n) \leq$ actual path cost



For Consistent

$h(A) < A\text{-}B + h(B)$

$9 < 5 + 1$

$9 < 6$ No

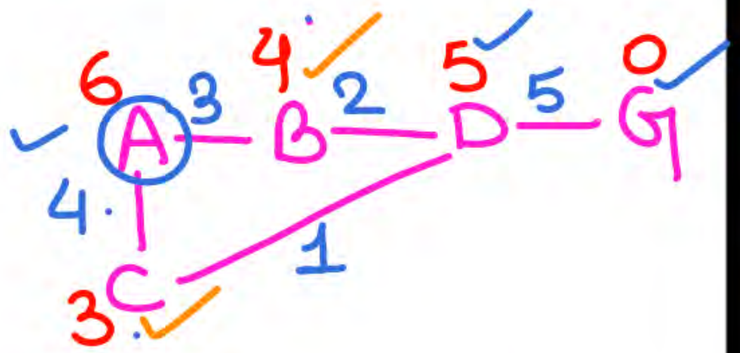
Type of Heuristic values

- Consistency implies admissibility, but the reverse is not always true.
- A consistent heuristic ensures that once a node has been expanded in A*, it will never need to be revisited, which makes the search more efficient.

A* will be efficient & optimal Result.

Given the following costs and heuristic values, is the heuristic admissible?

- **Graph edges (with actual costs):**
  - A to B: cost = 3
  - A to C: cost = 4
  - B to D: cost = 2
  - C to D: cost = 1
  - D to G (Goal): cost = 5
- **Heuristic values $h(x)$:**
  - $h(A) = 6$
  - $h(B) = 4$
  - $h(C) = 3$
  - $h(D) = 5$
  - $h(G) = 0$ $h(G)=0$



a) Consistent
b) Admissible
c) C+ admissible
d) none

$h(A) < h(B) + A - B$

$6 \leq 4+3$ Yes

$h(A) \leq h(C) + A - C$

$6 \leq 7$ Yes

$h(B) \leq h(D) + B - D$

$4 \leq 2+5$

$h(C) \leq h(D) + C - D$

$3 \leq 5+1$

$h(D) \leq D - G + h(G)$

$5 \leq 5+0$

Informed search : A* search Algorithm

Overview

- A Search* is one of the most widely used search algorithms in Artificial Intelligence for pathfinding and graph traversal.
- It combines the strengths of Dijkstra's Algorithm (which finds the shortest path) and Greedy Best-First Search (which uses a heuristic to guide the search).

Informed search : A* search Algorithm

- GBFS ⇒ look @ the h(n)
- A* ⇒ (h(n)+ weights)

Basic Concepts

Heuristics and Cost Functions

- Heuristic Function (h(n)): An estimate of the cost from node n to the goal.
- Cost Function (g(n)): The actual cost from the start node to node n
- Evaluation Function (f(n)): The sum of the heuristic and cost functions: f(n)=g(n)+h(n)

Informed search : A* search Algorithm

Here we create 2 list Open and closed list…

no need
we follow UCS wala method.

Informed search : A* search Algorithm

No need

A* Search Algorithm

Step-by-Step Process

- Initialize: Place the start node S in the open list with f(S)=g(S)+h(S).
- Expand: Remove the node n with the lowest f(n) from the open list and add it to the closed list.
- Goal Check: If n is the goal node G, the search is complete.
- Generate Successors: For each neighboring node of n:
- Calculate g(n), h(n), and f(n).
- If the node is not in the open or closed list, add it to the open list.
- If it's already in the open list with a higher f(n), update it.
- Repeat: Continue until the goal is found or the open list is empty.

Consistent heuristic

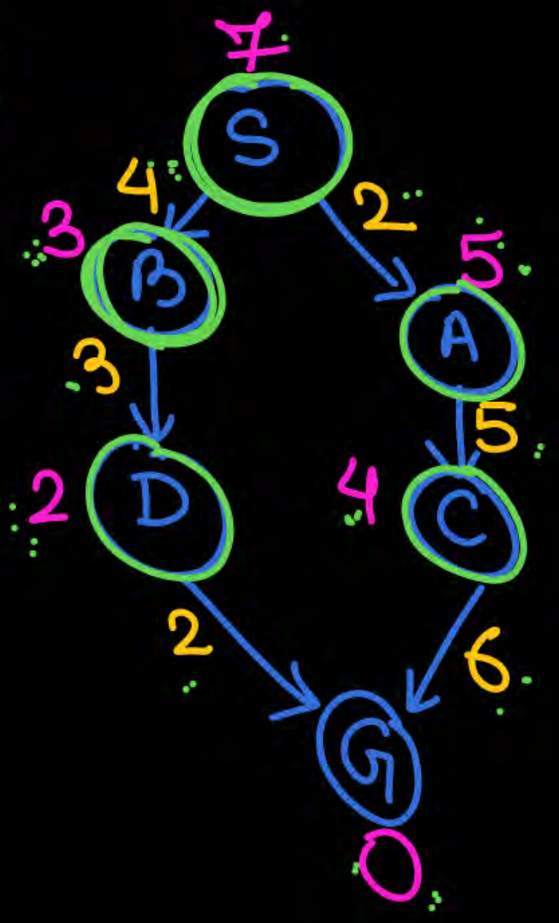
- Heuristic values $h(n)$:
  - $h(S) = 7$
  - $h(A) = 5$
  - $h(B) = 3$
  - $h(C) = 4$
  - $h(D) = 2$
  - $h(G) = 0$

- Nodes: S, A, B, C, D, G
- Edges and costs:
  - $S \to A$: 2
  - $S \to B$: 4
  - $A \to C$: 5
  - $B \to D$: 3
  - $C \to G$: 6
  - $D \to G$: 2

$S \longrightarrow G \Rightarrow 9$

Cost

	S	A	B	D
S	0	0	0	0
A	7	7	7	7
B	7	7	7	7
C	∞	11	11	11
D	∞	∞	9	9
G	∞	∞	∞	9



$S \leftarrow B \leftarrow D \leftarrow G$

Time & space Complexity

in worst-case scenario we have to visit all nodes

In graph ⇒ $T.C = O(b^d)$

$S.C = O(b^d)$

Time and space complexity

Time Complexity

The time complexity of A* largely depends on the heuristic function and the structure of the search space. In the worst case, the time complexity is **O(b^d)**, where:

- **b** is the branching factor (the average number of successors per state).
- **d** is the depth of the shallowest solution.

However, if the heuristic function is good (i.e., it is admissible and consistent), the time complexity can be significantly reduced, though it remains exponential in the worst case.

Time and space complexity

Space Complexity

The space complexity of A* is generally **O(b^d)** as well. A* keeps all generated nodes in memory, which can lead to high space usage, particularly in large or complex search spaces. This is because it stores the open list (the set of nodes that have been generated but not yet expanded) and the closed list (the set of nodes that have been fully explored).

Advantage

- Completeness
- Optimal Solution
- efficiency depend on heuristic fxn
- UCS: slow bcoz it visit all nodes, it find all possible path
  GBFS: not optimal, but fast.

this A* Combine advantage of both.

disadvantage

- optimal Solution but depend on quality of heuristic fxn.
- It may be not usefull for large graph/trees
- too much Calculation
- Slow wrt GBFS

Advantages

- **Optimality:** A* is guaranteed to find the least-cost path to the goal if the heuristic used is admissible (never overestimates the true cost) and consistent (satisfies the triangle inequality).
- **Efficiency:** A* is generally more efficient than uninformed search algorithms like Dijkstra's algorithm because it uses heuristics to guide the search. This often results in exploring fewer nodes, reducing the overall computation.
- **Flexibility:** A* can be adapted to various problems by changing the heuristic function. This flexibility makes it applicable to a wide range of scenarios, from simple grid navigation to complex graph-based pathfinding.
- **Combines Advantages of Greedy and Uniform-Cost Search:** A* combines the strengths of greedy best-first search (which is fast but not always optimal) and uniform-cost search (which is optimal but can be slow) by balancing exploration and exploitation.

Disadvantages

- High Memory Usage: A* requires storing all generated nodes in memory, which can lead to high memory consumption, especially in large search spaces. This makes it less suitable for problems with very large state spaces.
- Computationally Intensive: In the worst case, A* can be computationally expensive, especially if the branching factor is high or the heuristic is not well-optimized. The algorithm's time complexity is exponential in the worst case.
- Heuristic Dependency: The performance of A* heavily depends on the quality of the heuristic. If the heuristic is poorly chosen, the algorithm can degrade to a less efficient search, potentially exploring many unnecessary nodes.
- Not Always Practical for Real-Time Applications: Due to its potential high time and space complexity, A* may not be suitable for real-time applications where quick decisions are necessary.

A* $f(n) = \underbrace{\text{Cost}}_{\text{weight}} + h(n)$

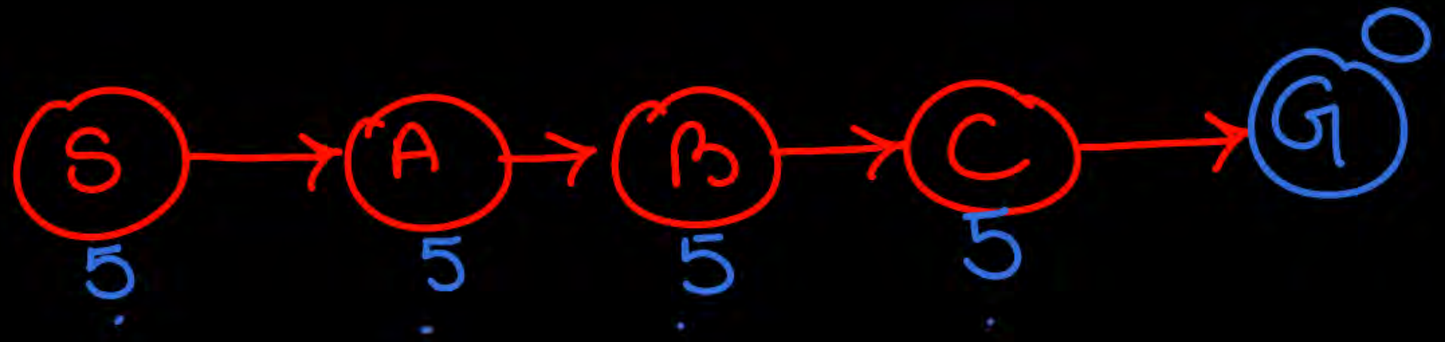
Zero

$S \xrightarrow{4} A \xrightarrow{3} B \xrightarrow{5} G$

$\Rightarrow \underline{12+0} \Rightarrow 12$

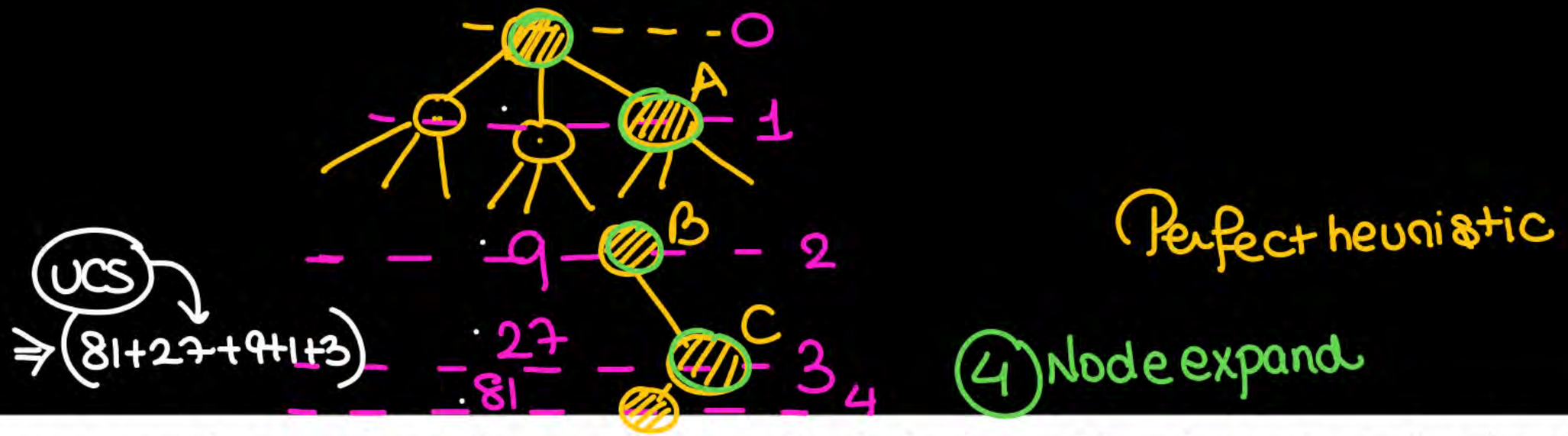
Consider a graph where the actual cost from the start node S to node A is 4, from node A to node B is 3, and from node B to the goal node G is 5. The heuristic estimates $h(n)$ are as follows: $h(A) = 7$, $h(B) = 2$, $h(G) = 0$. What is the estimated cost-to-goal $f(G)$ at node G when reached by A* search?

$h(G) + w(G)$



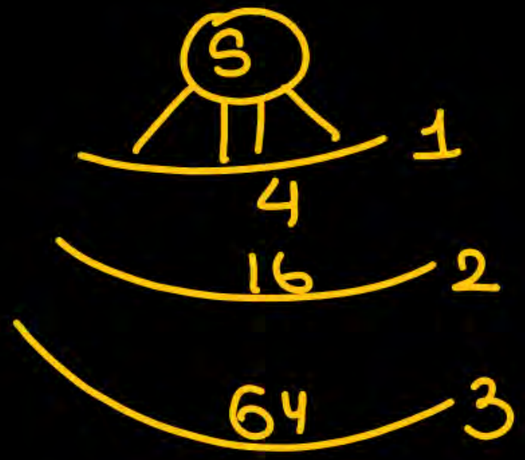
GBFS selects the node with the lowest heuristic value first. If the path found by GBFS from the start node S to the goal node G has a total heuristic sum of 20, and each expanded node contributes an equal heuristic value of 5, how many nodes were expanded?

4



Given a search tree where A* search is applied, the branching factor is 3, the depth of the optimal solution is 4, and the heuristic is perfect (i.e., it always predicts the exact remaining cost). How many nodes are expanded by A* in this tree?

$S \rightarrow A \rightarrow B \rightarrow C \rightarrow D$

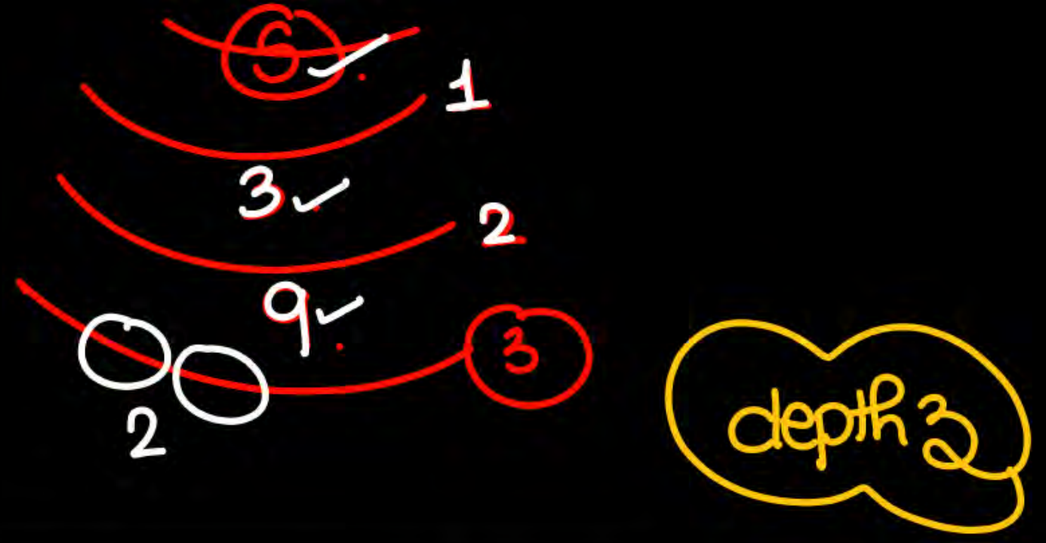


BFS

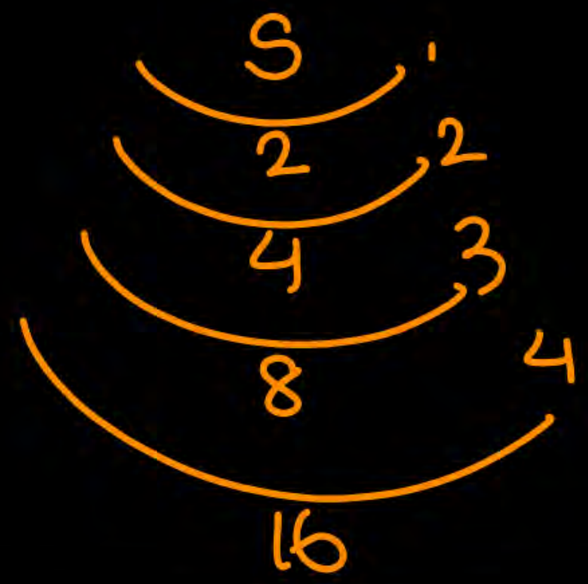
A BFS is performed on a tree where the branching factor is 4, and the goal node is at depth 3. What is the total number of nodes expanded before the goal is found?

$64+16+4+1$

85

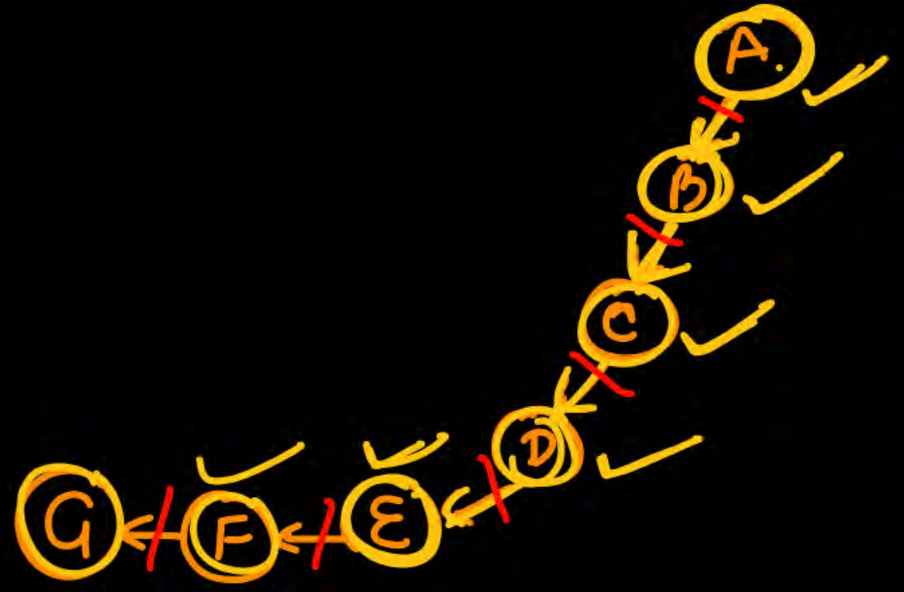


BFS explores all nodes at the present depth level before moving on to nodes at the next depth level. If the BFS algorithm expands 15 nodes and the branching factor of the graph is 3, what is the depth of the goal node?



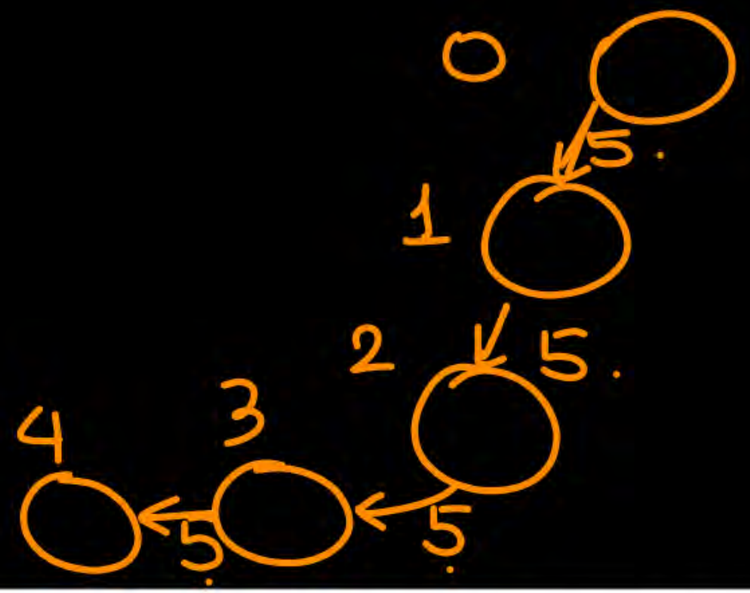
In a DFS on a tree with a branching factor of 2 and depth 4, what is the maximum number of nodes that the algorithm might expand before finding the goal node at the last depth?

$16+8+4+2+1=31$



A UCS is applied to a graph where each edge has a uniform cost of 3 units. If the search finds the optimal path by expanding 6 nodes, what is the total cost of the path found?

$6 \times 3 =$ 18



In a UCS, if the branching factor is 3 and each edge has a cost of 5 units, and the goal is located at a depth of 4, what is the minimum cost to reach the goal?

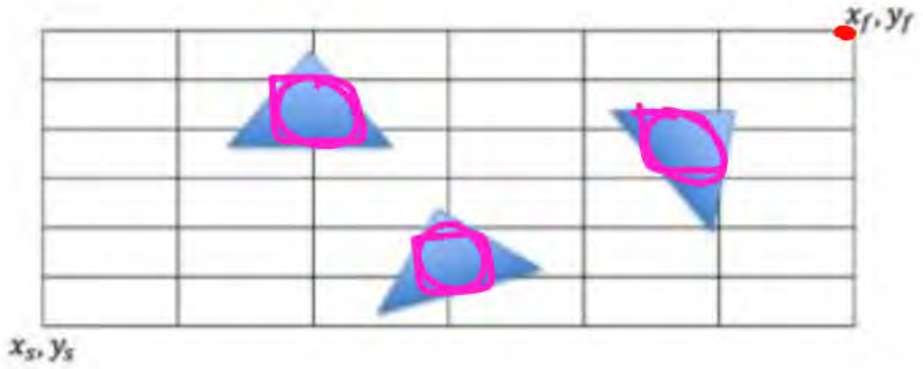
20

Consider the grid world shown in the figure below. An agent is planning to move from the starting location (x_s, y_s) to the final location (x_f, y_f). The obstacles along the path are triangular in form. Consider the following heuristic functions to conduct A* search.

- h_c assumes the obstacles are the smallest circles circumscribing the triangles.
- h_r assumes the obstacles are smallest rectangles circumscribing the triangles.
- h_c' assumes the obstacles are largest circles inscribed in the triangles.
- h_r' assumes the obstacles are largest rectangles inscribed the triangles.

Which of the following statement(s) is(are) true?

a) hc is admissible
b) hr " "
c) hc'
d) hr' " "

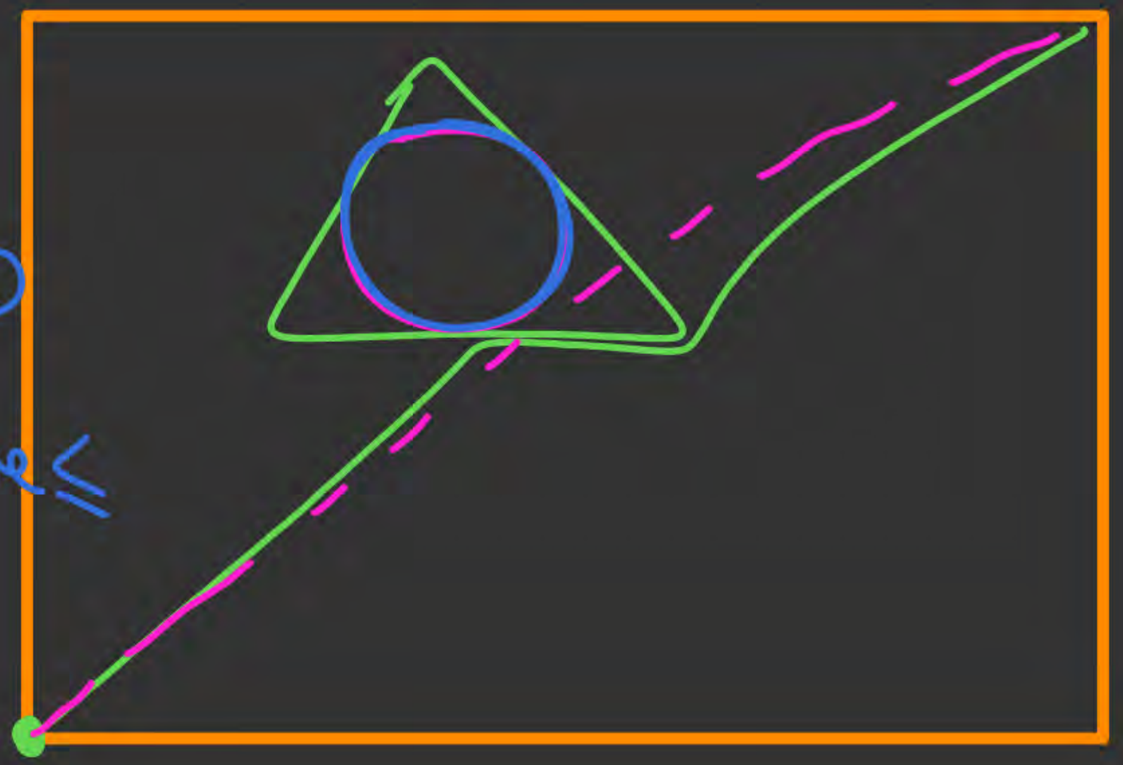


(A)	h_c is an admissible heuristic
(B)	h_r is an admissible heuristic
(C)	h_c' is an admissible heuristic
(D)	h_r' is an admissible heuristic

Small obstacle

$\Downarrow$

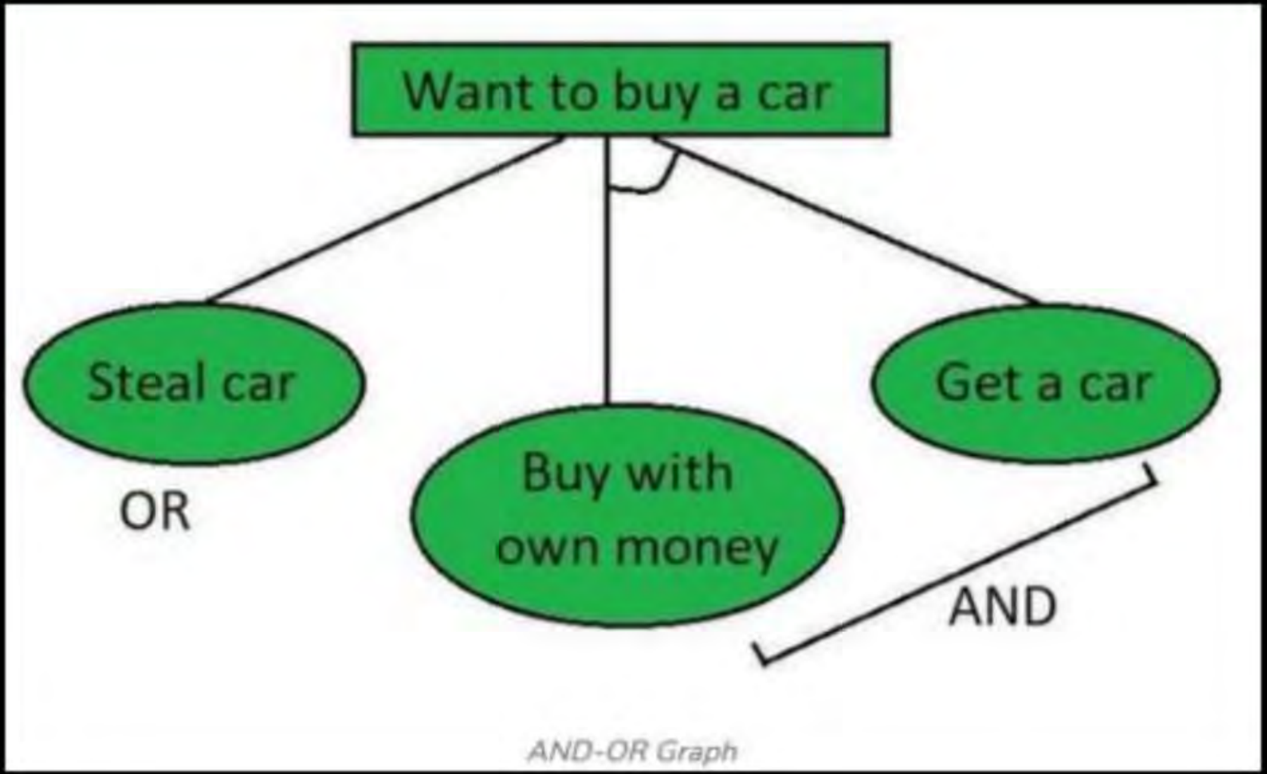
then heuristic value $\leq$ actual cost.



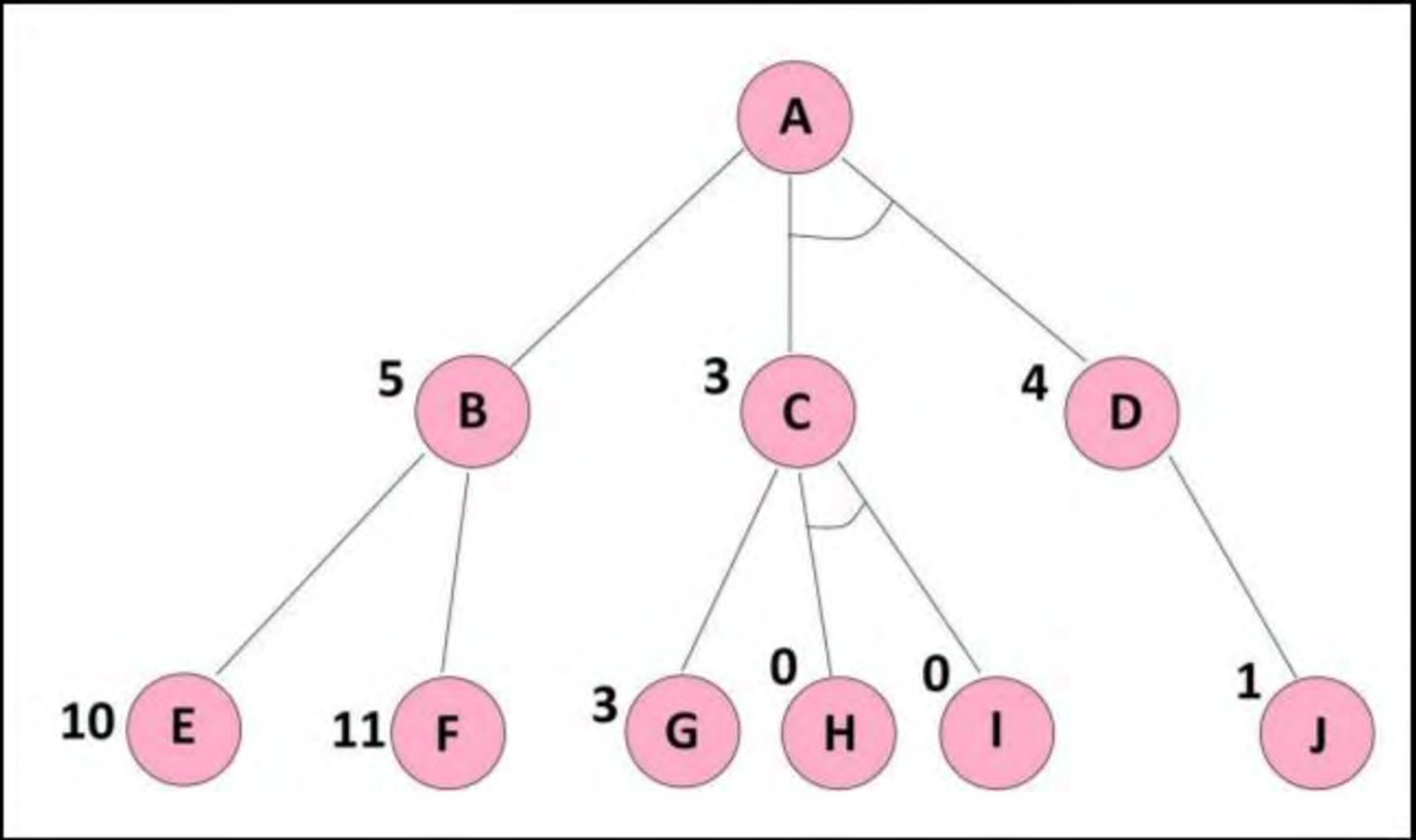
AO* algorithm – Artificial intelligence

- The AO* method divides any given difficult problem into a smaller group of problems that are then resolved using the AND-OR graph concept.
- AND OR graphs are specialized graphs that are used in problems that can be divided into smaller problems.
- The AND side of the graph represents a set of tasks that must be completed to achieve the main goal, while the OR side of the graph represents different methods for accomplishing the same main goal.

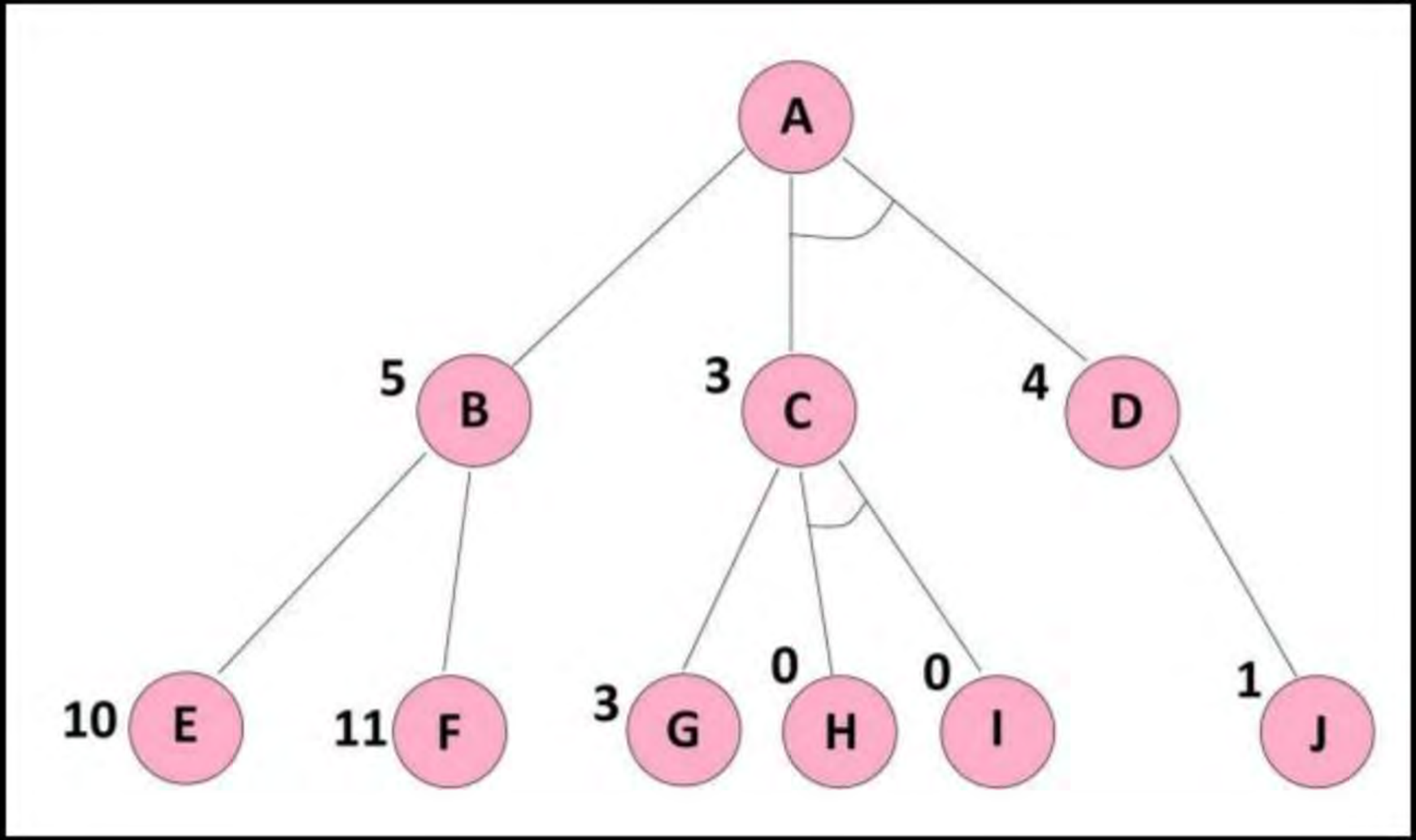
AO* algorithm – Artificial intelligence



- Forward Propagation

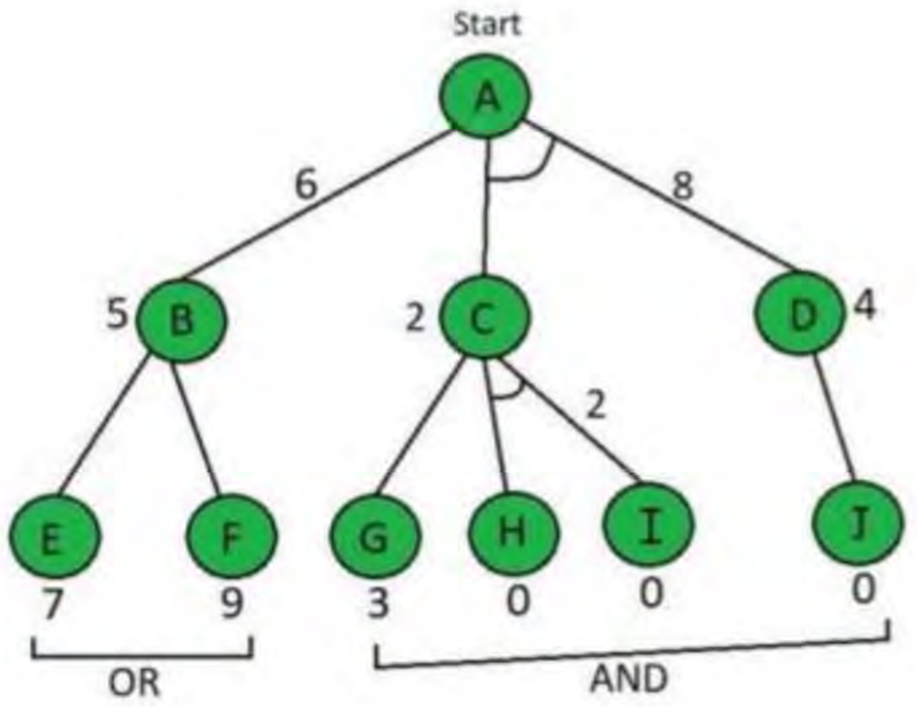


- Reaching the Last Level and Back Propagation

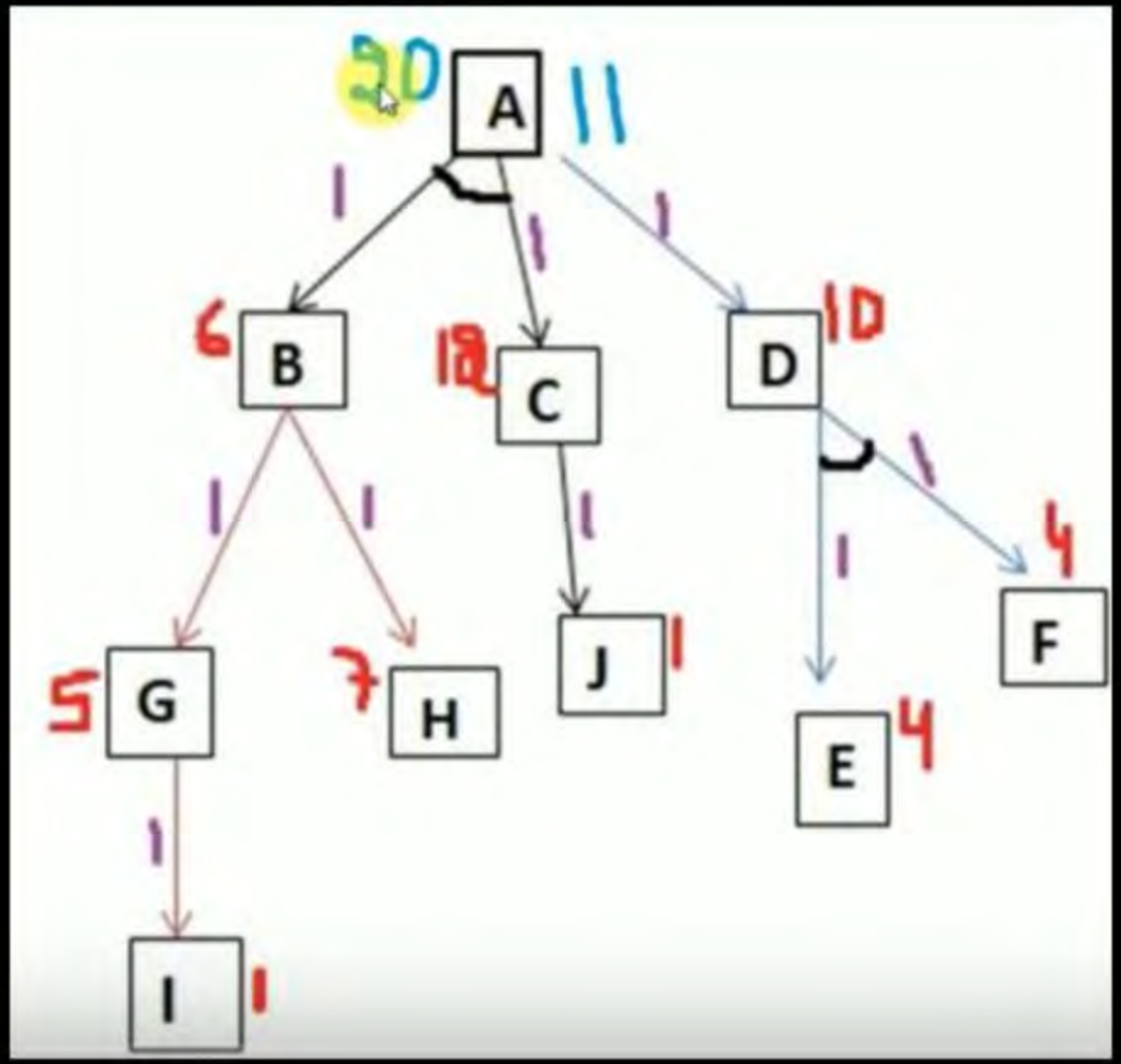


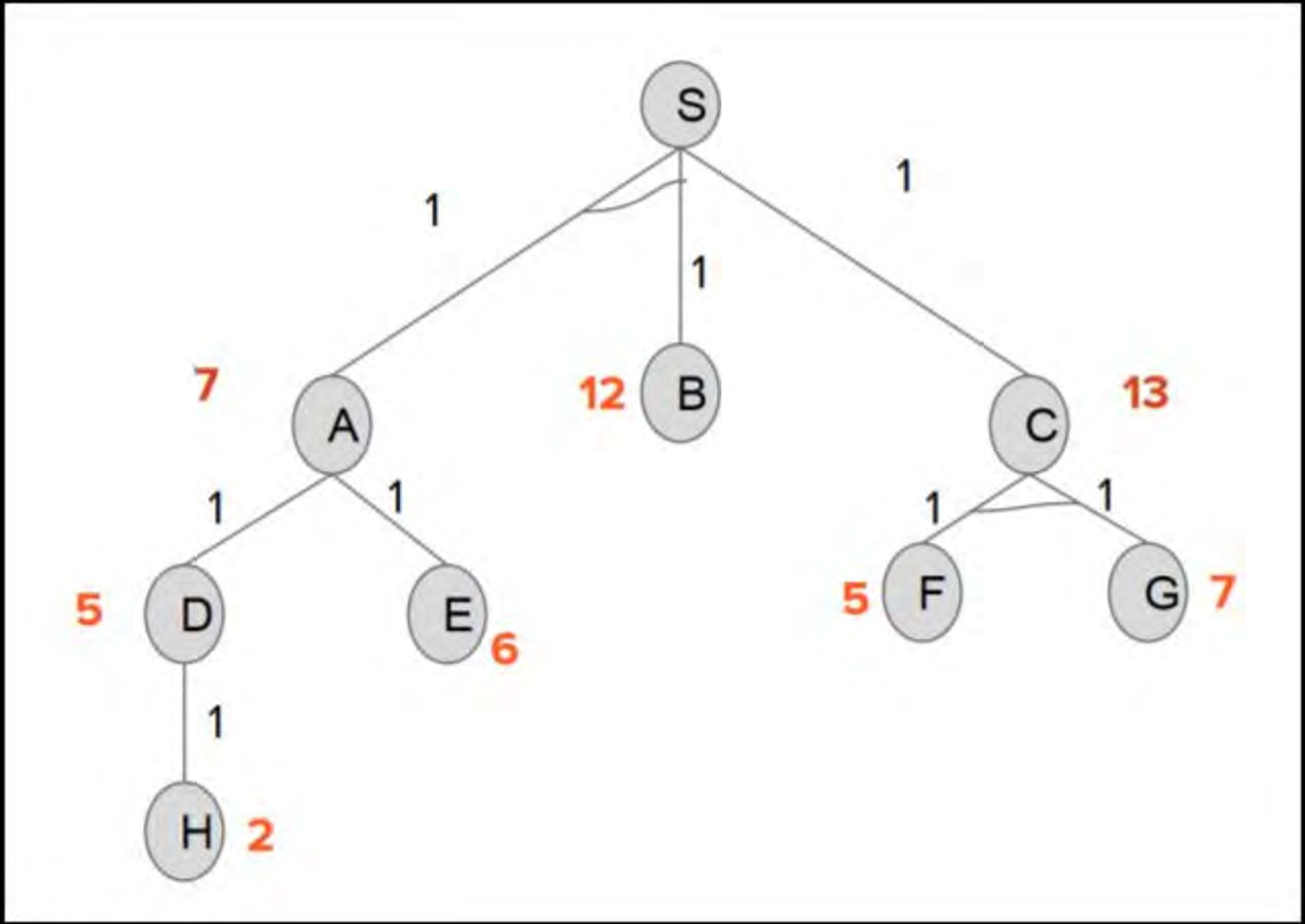
AO* algorithm – Artificial intelligence

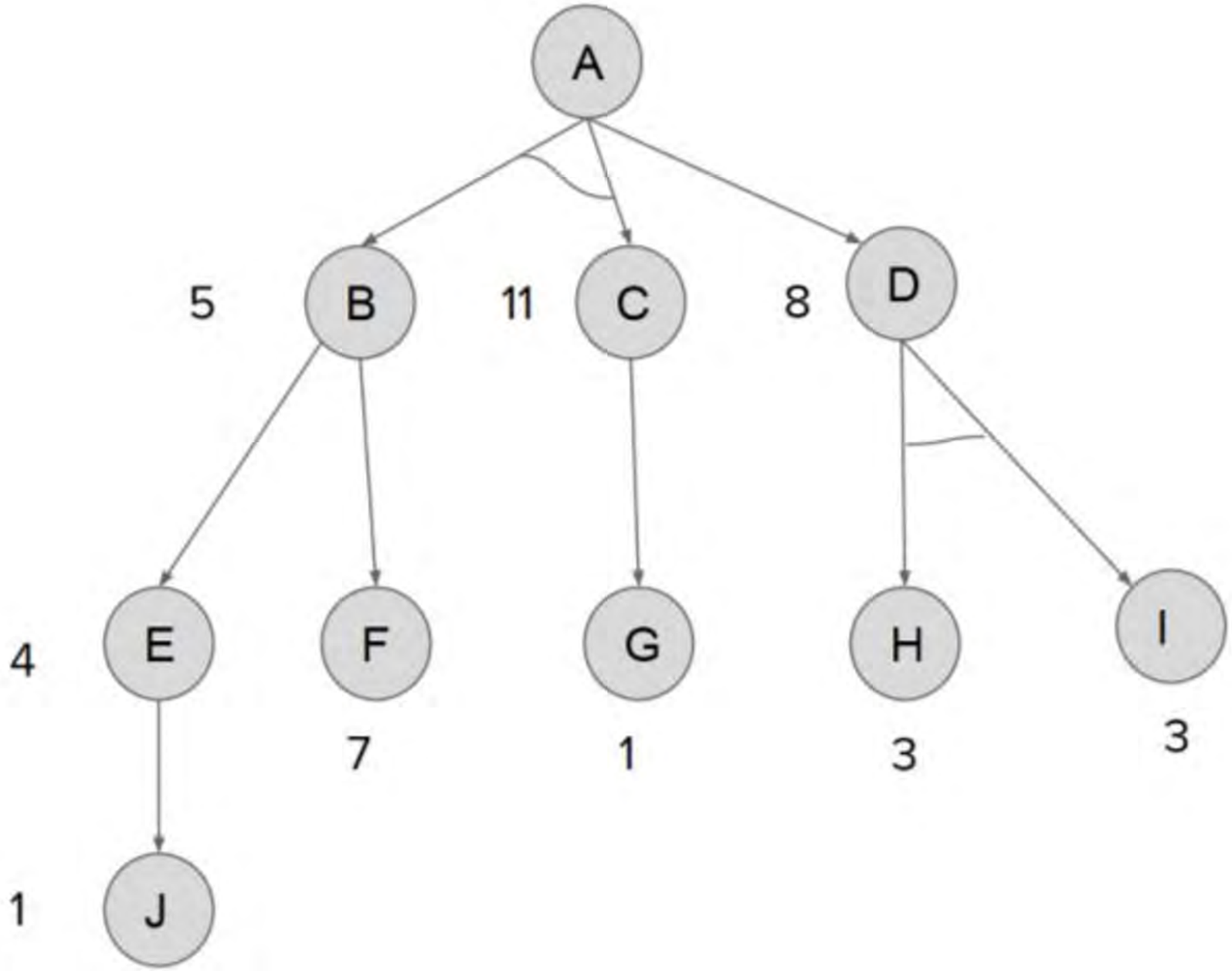
- In the above figure, the buying of a car may be broken down into smaller problems or tasks that can be accomplished to achieve the main goal in the above figure, which is an example of a simple AND-OR graph. The other task is to either steal a car that will help us accomplish the main goal or use your own money to purchase a car that will accomplish the main goal.
- The AND symbol is used to indicate the AND part of the graphs, which refers to the need that all subproblems containing the AND to be resolved before the preceding node or issue may be finished.
- The start state and the target state are already known in the knowledge-based search strategy known as the AO* algorithm, and the best path is identified by heuristics. The informed search technique considerably reduces the algorithm's time complexity. The AO* algorithm is far more effective in searching AND-OR trees than the A* algorithm.



AO Algorithm – Question tree*


20
A
11
1
1
1
6
B
12
C
D
10
1
1
1
1
1
4
F
5
G
7
H
J
1
E
4
1
I
1


S
1
1
1
7
A
12
B
C
13
1
1
1
1
5
D
E
6
5
F
G
7
1
H
2


A
5
B
11
C
8
D
4
E
F
G
H
I
7
1
3
3
1
J

THANK - YOU